Chapter 11. Threaded Java

11.0 Introduction

We live in a world of multiple activities. A person may be talking on the phone while doodling or reading an email. A multifunction office machine may scan one fax while receiving another and printing a document from somebody's computer. We expect the GUI programs we use to be able to respond to a menu while updating the screen. But ordinary computer programs can do only one thing at a time. The conventional computer programming model—that of writing one statement after another, punctuated by repetitive loops and binary decision making—is sequential at heart.

Sequential processing is straightforward but not as efficient as it could be. To enhance performance, Java offers *threading*, the capability to handle multiple flows of control within a single application or process. Java provides thread support and, in fact, requires threads: the Java runtime itself is inherently multithreaded. For example, window system action handling and Java's garbage collection—that miracle that lets us avoid having to free everything we allocate, as others must do when working in languages at or below C level—run in separate threads.

Just as multitasking allows a single operating system to give the appearance of running more than one program at the same time on a single-processor computer, multithreading can allow a single program or process to give the appearance of working on more than one thing at the same time. Multithreading leads to more interactive graphics and more responsive GUI applications (the program can draw in a window while responding to a menu, with both activities occurring more or less independently), more reliable network servers (if one client does something wrong, the server continues communicating with the others), and so on.

Note that I did not say "multiprocessing;" the term multi-tasking is sometimes erroneously called multiprocessing, but the latter term in fact refers to a different issue: it's the case of two or more CPUs running under a single operating system. Multiprocessing per se is nothing new: IBM mainframes did it in the 1970s, Sun SPARCstations did it in the 1980s, and Intel PCs did it starting in the 1990s. Since the mid-2010s, it has become

increasingly hard to buy a single-processor computer packaged inside anything larger than a wristwatch. True multiprocessing allows you to have more than one process running concurrently on more than one CPU. Java's support for threading includes multiprocessing, as long as the operating system supports it. Consult your system documentation for details.

Though most modern operating systems provide threads, Java was the first mainstream programming language to have intrinsic support for threaded operations built right into the language. The semantics of `java.lang.Object`, of which all objects are instances, includes the notion of monitor locking of objects, and some methods (`notify`, `notifyAll`, `wait`) are meaningful only in the context of a multithreaded application. Java also has language keywords such as `synchronized` to control the behavior of threaded applications.

In modern Java there are two types of threads, *platform threads* and *virtual threads*. As the name implies, the former are coupled one-to-one with platform (operating system) threads, which the latter are multiplexed among a small pool of platform threads.

Threads can have a name associated with them, and a priority. Platform threads can also be *daemon* or *non-daemon*. When a normal Java application is started, there is one non-daemon thread (e.g., the thread that called the application's `main()` method from the `java` command). The application will shut down when all running non-daemon threads have terminated. Non-daemon threads are used, for example, to handle mouse and keyboard events in a GUI application, such that the `main()` method can end or `return` without causing the application to terminate. Platform threads have a thread priority and are members of a thread group. Virtual threads are always daemon threads. Recipe 11.2 provides coverage specific to virtual threads.

Now that the world has had years of experience with threaded Java, experts have started building better ways of writing threaded applications. The concurrency utilities, originally specified in JSR-166 and included in all modern Java releases, are heavily based on the util.concurrent package by Professor Doug Lea of the Computer Science Department at the SUNY Oswego. This package aims to do for the difficulties of threading what the Collections Framework (see Chapter 7) did for structuring data. This is no small undertaking, but they pulled it off.

The java.util.concurrent package includes several main sections:

- `Executor`s, thread pools (`ExecutorService`s), and `Future`s/`CompletableFuture`s
- `Queue`s and `BlockingQueue`s

- Locks and conditions, with JVM support for faster locking and unlocking
- Synchronizers, including `Semaphore`s and `Barrier`s
- Atomic variables

In this chapter, I will focus on the first set of these, thread pools and `Future`s.

An implementation of the `Executor` interface is, as the name implies, a class that can execute code for you. The code to be executed can be the familiar `Runnable` or a new interface `Callable`. One common type of `Executor` is a *thread pool*. The `Future` interface represents the future state of something that has been started; it has methods to wait until the result is ready. A `CompletableFuture` is an implementation of `Future` that adds many additional methods for chaining `CompletableFuture`s and post-applied methods. These brief definitions are simplifications. Addressing all the issues is beyond the scope of this chapter, but the chapter provides examples of the most important parts.

11.1 Running Code in a Different Thread

Problem

You need to write a threaded application. Threads allow your program to do more than one thing at a time, such as servicing multiple users, multiple connections, or in a desktop application, responding to UI events while loading a file, or in a video game, moving multiple objects around the screen.

Solution

Write code that implements `Runnable`; pass it to an `Executor`, or instantiate a `Thread` and start it.

Discussion

There are several ways to implement threading, and they all require you to implement the `Runnable` or `Callable` interface. `Runnable` has only one method, and it returns no value; this is its signature:

```
public interface java.lang.Runnable {
  public abstract void run();
```

```
}
```

Similarly, `Callable` has only one method, but the `call()` method returns a specific type so the interface has a type parameter (`V` here, for "value"):

```
public interface java.util.concurrent.Callable<V> {
  public abstract V call() throws Exception;
}
```

You must provide an implementation of the `run()` or `call()` method. There is nothing special to this method; it's an ordinary method and you could call it yourself. But if you did, what then? There wouldn't be the special magic that launches it as an independent flow of control, so it wouldn't run concurrently with your main program or flow of control. For this, you need to invoke the magic of thread creation.

The original approach to using threads, no longer generally recommended, is to create `Thread` objects directly and call their `start()` method, which would cause the thread to call the `run()` method after the new thread had been initialized. There was no support for the `Callable` interface in the original threads model. You create threads by doing one of the following:

- Subclass `java.lang.Thread` (which implements `Runnable`) and override the `run()` method.
- Create your `Runnable` and pass it into the `Thread` constructor.
- With Java 8+, you can use a lambda expression to implement `Runnable`, as shown in the introdcution to Chapter 9.

This approach is not recommended because of issues such as performance (`Thread` objects are expensive to create and tear down, and a thread is unusable once its `run()` method returns). This book no longer shows examples of doing so. There are some examples in the online source, in the *threads* directory; see especially *ThreadsDemo4*.

When you need to stop a thread, don't use the `Thread.stop()` method; instead, use a `boolean` tested at the top of the main loop in the `run()` method. That's because the `stop()` method is so drastic that it can never be made to behave reliably in a program with multiple active threads. That is why, when you try to use it, the compiler will generate deprecation warnings. The recommended method (for threads with a loop) is to use a `boolean` variable in the main loop of the `run()` method. Example 11-1 prints a message endlessly until its `shutDown()` method is called; it then sets the controlling variable

`done` to `false`, which terminates the loop. This causes the `run()` method to return, ending its processing.

Example 11-1. main/src/main/java/threads/StopBoolean.java

```
public class StopBoolean {

 // Must be volatile to ensure changes visible to other threads.
 protected volatile boolean done = false;

 Runnable r = () -> {
  while (!done) {
   System.out.println("StopBoolean running");
   try {
    Thread.sleep(720);
   } catch (InterruptedException ex) {
    // nothing to do
   }
  }
  System.out.println("StopBoolean finished.");
 };

 public void shutDown() {
  System.out.println("Shutting down...");
  done = true;
 }

 public void doDemo() throws InterruptedException {
  ExecutorService pool = Executors.newSingleThreadExecutor();
  pool.submit(r);
  Thread.sleep(1000*5);
  shutDown();
  pool.shutdown();
  pool.awaitTermination(2, TimeUnit.SECONDS);
 }

 public static void main(String[] args) throws InterruptedException {
  new StopBoolean().doDemo();
 }
}
```

Instead of starting and stopping threads explicitly, the recommended way to perform threaded operations is to use a thread pool, the implementation of which is the java.util.concurrent package's `ExecutorService`. An `ExecutorService` is, as its name im-

plies, a service class that can execute code for you. The code to be executed can be in a `Runnable` or a `Callable`. You obtain an `ExecutorService` by invoking a factory method on the `Executors` class. The code in Example 11-2 shows a simple example of a thread pool.

Example 11-2. main/src/main/java/threads/ThreadPoolDemo.java

```
    final ExecutorService pool = Executors.newFixedThreadPool(HOWMANY);
    List<Future<Integer>> futures = new ArrayList<>(HOWMANY);
    for (int i = 0; i < HOWMANY; i++) {
      Future<Integer> f = pool.submit(new DemoRunnable(i));
      System.out.println("Got 'Future' of type " + f.getClass());
      futures.add(f);
    }
    Thread.sleep(3 * 1000);
    done = true;
    for (Future<Integer> f : futures) {
      System.out.println("Result " + f.get());
    }
    pool.shutdown();
  }

  static class DemoRunnable implements Callable<Integer> {
    int time, numRuns;
    DemoRunnable(int t) { time = t; }

    @Override public Integer call() {
      while (!done) {
        System.out.println("Running " + Thread.currentThread());
        ++numRuns;
      }
      System.out.println("Stopping " + this);
      return numRuns;
    }
  }
```

This will print a series of lines like the following, showing the threads running interspersed:

```
Running Thread[pool-1-thread-3,5,main]
Running Thread[pool-1-thread-3,5,main]
Running Thread[pool-1-thread-1,5,main]
Running Thread[pool-1-thread-1,5,main]
```

```
...
Stopping threads.ThreadPoolDemo$DemoRunnable@2bdd2b00
Stopping threads.ThreadPoolDemo$DemoRunnable@3e413b00
Result 120056
Result 123719
...
```

Note that there are several submission methods, the first in the parent interface `Executor` and two more in `ExecutorService`:

```
public void execute(Runnable);
public Future<T> submit(Callable<T>);
public Future<T> submit(Runnable);
```

That is, `execute()` takes a `Runnable` and returns nothing, while the `submit()` methods both return a `Future<T>` (for the method `submit(Runnable)`, the type parameter `T` is always `java.lang.Void`).

When you are finished with the thread pool, you should call its `shutDown()` method.

`Future` is a software analogue of the claim ticket you are given when you take laundry in to a dry cleaning service or take some item in to be repaired. It's an interface representing a claim on some deliverable that either will or will not be ready at some point in time, and that will hopefully change from not ready to ready "sometime." The following shows the important methods of the `Future` interface:

```
public interface java.util.concurrent.Future<V> {
   public abstract boolean isDone();
   public abstract V get() throws InterruptedException,
      java.util.concurrent.ExecutionException;
   public abstract V get(long timeout, java.util.concurrent.TimeUnit)
      throws InterruptedException,
      java.util.concurrent.ExecutionException,
      java.util.concurrent.TimeoutException;
   public abstract boolean cancel(boolean);
   public abstract boolean isCancelled();
}
```

The purpose of each method in this interface is shown here:

`isDone()`

Returns true if the operation that will deliver the result has completed.

`get()`

Will return the deliverable immediately if `isDone()` is true; else will block indefinitely until it becomes true.

`get(long, TimeUnit)`

Will return the deliverable immediately if `isDone()` is true, blocking until it becomes true, or until the specified time has elapsed, in which case it will throw a `TimeoutException`. `TimeUnit` is an enum in `java.util.concurrent` with values for days, hours, minutes, seconds, etc.

`cancel(boolean)`

Will cancel the operation if it hasn't started (if the `boolean` is false) or even if it is in process (if the `boolean` is true).

`isCancelled()`

Returns true if the operation has been canceled.

`Future` is commonly returned from a thread pool `execute()` operation, as shown in Example 11-3.

Example 11-3. main/src/main/java/threads/FutureFromThreadpool.java

```
double d = 2;
Callable<Double> computeTotal = () -> d + d;
Future<Double> future = threadPool.submit(computeTotal);
while (!future.isDone()) {
 Thread.sleep(100);
}
double value = future.get();
process(value);
threadPool.shutdown();
```

There are many classes implementing `Future` in various parts of Java SE and Jakarta. The most general and powerful is `CompletableFuture<V>`, so called because you can control it by calling a `complete(V)` method at any time. This has far too many public methods (120) for a complete treatment here. In fairness, the number of methods is high because many methods can accept either a `Runnable` or a `Callable`, and many have multiple overloads (a plain one, one with `Async` appended to the name, and one with `Async` that lets you provide the `Executor`). I'll show examples of a few of these shortly.

You can create an empty `CompletableFuture` by calling a no-argument constructor, making this available to some calling code, and calling its `complete()` method when you have a result:

```
CompletableFuture<Integer> cf = new CompletableFuture<>();
// Do some work
cf.complete();
```

Many of the more interesting methods in `CompletableFuture` have to do with chaining operations. First, you can specify a function to be invoked automatically when the result is ready:

```
public CompletableFuture<Void> thenRun(java.lang.Runnable);
public CompletableFuture<Void> thenRunAsync(java.lang.Runnable);
public CompletableFuture<Void> thenRunAsync(java.lang.Runnable, Executor);
public <U> CompletableFuture<U> thenApply(
```

```
    Function<? super T,
      ? extends U>);
  public <U> CompletableFuture<U> thenApplyAsync(
    Function<? super T,
      ? extends U>);
  public <U> CompletableFuture<U> thenApplyAsync(
    Function<? super T,
      ? extends U>, Executor);
  public CompletableFuture<Void> thenAccept(Consumer<? super T>);
  public CompletableFuture<Void> thenAcceptAsync(Consumer<? super T>);
  public CompletableFuture<Void> thenAcceptAsync(
    Consumer<? super T>, Executor);
```

These methods will invoke the given `Runnable`, `Consumer`, or `Function` after the `Future` is completed. Each of these exists in the three forms as mentioned previously. The first method will run it on the same thread as the main task. The second method will run it in a default executor. The third allows you to provide your own `Executor`. Example 11-4 is a simple demo of creating a `CompletableFuture` and giving it a `thenApply` method call and a final `run` method, both of which don't fire until the `Future` is completed.

Example 11-4. main/src/main/java/threads/CompletableFutureSimple.java

```
class CompletableFutureSimple {
 static String twice(String x) { return x + ' ' + x; }

 public static void main(String[] args) {
  CompletableFuture<String> cf = new CompletableFuture<>();
  cf.thenApply(x -> twice(x))
   .thenAccept(x -> System.out.println(x));
  // Possibly some computation going on here... Then:
  cf.complete("Hello");
 }
}
```

The online source includes *CompletableFutureDemo.java*, which offers some more sophisticated examples.

11.2 Using Virtual Threads for Better

Performance

Problem

You want to experience faster performance of a multithreaded, I/O-bound process.

Solution

Use virtual threads.

Discussion

The original threading mechanism, standard since the early days of Java, had one major drawback: overhead. Each Java thread was tied to a single OS-level thread. When that thread was blocked, for example when doing input/output, the Java thread could not progress. Actually, some of the very early JVMs used a different model, called *green threads*, in which the Java threads were multiplexed onto system-level threads, initially because some of the target operating systems then in use didn't support application use of threads! This was abandoned after all the operating systems caught up. With virtual threads we see a return to this policy of multiplexing, but with two decades of experience in how to design it better. Now, the JVM will attach the virtual thread to an available OS-level thread. This has the consequence that a task running in a virtual thread may be attached to several different physical threads over its lifetime.

The nice thing about virtual threads is that, from a developer's point of view, they can most easily be used with what you already know. We mentioned earlier that thread pools are the recommended means of writing threaded code. That's still true with virtual threads. You just have to say:

```
ExecutorService pool = Executors.newVirtualThreadPerTaskExecutor();
```

And presto! Tasks you submit to `pool` will run on virtual threads.

Alternatively, if for some reason you don't want to use a thread pool, you can get virtual threads one at a time just by asking and providing a `Runnable`:

```
// Mainly for apps that only ever need one thread
Thread t = Thread.startVirtualThread(runnable);
```

Since the return type is `Thread` and there is no `Future` associated with a `Thread`, if you want to use a `Callable`, you must use the thread pool approach.

Both approaches are demonstrated in Example 11-5.

Example 11-5. main/src/main/java/threads/VirtualThreadsDemo.java

```
public static void main(String[] args) throws Exception {

 Runnable r = () -> {
  System.out.println("Hello from " + Thread.currentThread());
 };

 // Start vthreads with an ExecutorService
 try (var executor = Executors.newVirtualThreadPerTaskExecutor()) {
  IntStream.range(0, MAX).forEach(i -> executor.submit(r));
 }

 // Start vthreads directly
 IntStream.range(0,MAX).forEach(i -> Thread.startVirtualThread(r));

 Thread.sleep(100);
}
```

Note that `MAX` in the preceding example can be set to be quite large—you can create many thousands of virtual threads, whereas that many regular threads would probably bring the operating system to its (virtual) knees. An app with that many tasks happening at once would probably benefit from virtual threads.

Incidentally, when you want a `Callable` but have a `Runnable`, there are several overloads of a converter method in the `Executors` class, including:

```
static callable(Runnable task);
static <T> Callable<T> callable(Runnable task, T result);
```

These create a `Callable` wrapper that returns `null` in the first case or a value of the provided type in the second case.

NOTE

Not all types of programs will benefit from switching to virtual threads. Programs that have a lot of threads waiting for I/O operations to complete are the most likely to see improvements. Determining whether virtual threads are faster requires careful measurement with a significant amount of input.

We didn't take the time to compare the implementations, as you'd need a larger sample to see the difference. You might look at *main/src/main/java/lang/Timer.java* for a skeleton program for timing comparisons.

11.3 Rendezvous and Timeouts

Problem

You need to know whether something finished or whether it finished in a certain length of time.

Solution

Start it in its own thread and call its `join()` method with or without a timeout value.

Discussion

The `join()` method of the target thread is used to suspend the current thread until the target thread is finished (that is, returns from its `run()` method). This method is overloaded; a version with no arguments waits forever for the thread to terminate, whereas a version with arguments waits up to the specified time. For a basic example, I create (and start!) a simple thread that just reads from the console terminal, and the main thread simply waits for it. When I run the program, it looks like this:

```
ian $ java threads.Join
Starting
Joining
Reading
hello from standard input # blocking wait for me to type this line
Thread Finished.
Main Finished.
ian $
```

Example 11-6 lists the code for the `join()` demo.

Example 11-6. main/src/main/java/threads/Join.java

```
public class Join {
  public static void main(String[] args) {
    System.out.println("Starting");
    Thread t = Thread.startVirtualThread(() -> {
      System.out.println("Reading");
      try {
        System.in.read();
      } catch (java.io.IOException ex) {
        System.err.println(ex);
      }
      System.out.println("Thread Finished.");
    });
    System.out.println("Joining");
    try {
      t.join();
    } catch (InterruptedException ex) {
      // should not happen:
      System.out.println("Who dares interrupt my sleep?");
    }
    System.out.println("Main Finished.");
  }
}
```

This code uses a `Runnable` lambda (see Recipe 11.1) in `Thread t`.

11.4 Synchronizing Threads with the synchronized Keyword

Problem

You need to protect certain data from access by multiple threads.

Solution

Use the `synchronized` keyword on the method or code you wish to protect.

Discussion

This keyword specifies that only one thread at a time is allowed to run the given method (or any other synchronized method in the same class) in a given object instance (for static methods, only one thread is allowed to run the method at a time). You can synchronize methods or smaller blocks of code. It is easier and safer to synchronize entire methods, but this can be more costly in terms of blocking threads that could run. To do so, you can simply add the `synchronized` keyword on the method. For example, many of the methods of `Vector` (see Recipe 7.5) are synchronized in order to ensure that the vector does not become corrupted or give incorrect results when two threads update or retrieve from it at the same time.

Bear in mind that threads can be interrupted at almost any time, in which case control is given to another thread. Consider the case of two threads appending to a data structure at the same time. Let's suppose we have the same methods as `Vector`, but we're operating on a simple array. The `add()` method simply uses the current number of objects as an array index, then increments it:

```
public void add(Object obj) {
  data[max] = obj; ❶
  max = max + 1;  ❷
}
```

Threads A and B both want to call this method. Suppose that Thread A gets interrupted after ❶ but before ❷, and then Thread B gets to run.

❶ Thread B does ❶, overwriting the contents of `data[max]`; we've now lost all reference to the object that Thread A passed in!

❷ Thread B then increments `max` at ❷ and returns. Later, Thread A gets to run again; it resumes at ❷ and increments `max` past the last valid object. So not only have we lost an object, but we have an uninitialized reference in the array. This state of affairs is shown in Figure 11-1.

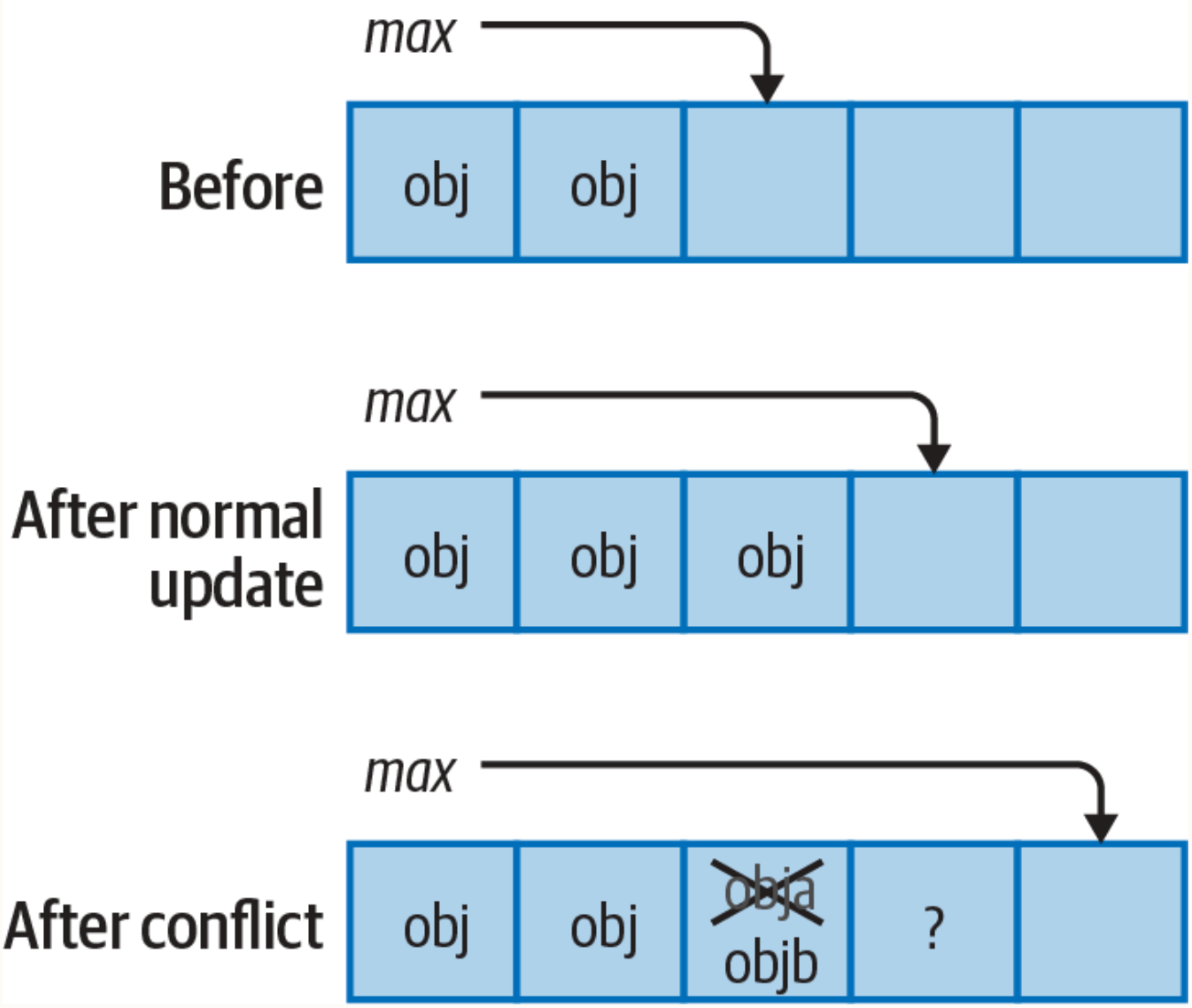


Figure 11-1. Non-thread-safe `add` method in operation: normal and failed updates

Now, you might think, "No problem, I'll just combine the two lines of code!":

```
data[max++] = obj;
```

As the game show host sometimes says, "Bzzzzt! Thanks for playing!" This change makes the code a bit shorter but has absolutely no effect on reliability. Interrupts don't happen conveniently on Java statement boundaries; they can happen between any of the many JVM instructions that correspond to your program. The code can still be interrupted after the store and before the increment. The only good solution is to use proper synchronization.

Making the method `synchronized` means that any invocations of it will wait if one thread has already started running the method:

```
public synchronized void add(Object obj) {
    ...
}
```

Any time you wish to synchronize some code, but not an entire method, use the `synchronized` keyword on an unnamed code block within a method, like this:

```
public void add(Object obj) {
   synchronized (someObject) {
      // this code will execute in one thread at a time
   }
}
```

The choice of which object to synchronize on is up to you. Sometimes it makes sense to synchronize on the object containing the code (e.g., `synchronized(this) {...}`) or using a `synchronized` method. For synchronizing access to an `ArrayList`, it would make sense to use the `ArrayList` instance, like this:

```
synchronized(myArrayList) {
    if (myArrayList.indexOf(someObject) != -1) {
       // do something with it.
    } else {
       create an object and add it...
   }
}
```

The code in Example 11-7 shows an array being accessed in a loop by two threads. Two versions of the accessing code are shown, `runGood` and `runBad`. Only the former is synchronized, and only the former is reliable. The code synchronizes on the array, so any code accessing the array while either thread is still running should also synchronize on the array.

Example 11-7. main/src/main/java/threads/ArrayAdding.java

```
public class ArrayAdding {
  private static final int HOWMANY = 1000;
   private static int[] array;
  private static ExecutorService pool = Executors.newFixedThreadPool(2);

   static Runnable runBad = () -> {
      for (int i = 0; i < array.length; i++) {
         array[i] = array[i] + i;
      }
   };
```

```
  static Runnable runGood = () -> {
    synchronized(array) {
      for (int i = 0; i < array.length; i++) {
        array[i] = array[i] + i;
      }
    }
  };

    public static void main(String[] args) throws Exception {
    process("runGood", runGood);
    process("runBad", runBad);
  }

  static void process(String name, Runnable run) throws Exception {
    System.out.println("Starting: " + name);
    array = new int[HOWMANY];
       var t1 = pool.submit(runBad);
       var t2 = pool.submit(runBad);
    t1.get();
    t2.get();
    for (int i = 0; i < array.length; i++) {
      if (array[i] != 2 * i) {
        System.out.printf("%d found at offset %d\n", array[i], i);
        return;     // A single failure crashes the rocket.
      }
    }
  }
}
```

Here are four runs, in which `runBad` fails twice, while `runGood` never fails:

```
$ java ArrayAdding.java
206881 found at offset 206881
Starting: runGood
Starting: runBad
$ java ArrayAdding.java
Starting: runGood
Starting: runBad
$ java ArrayAdding.java
Starting: runGood
Starting: runBad
468 found at offset 468
$ java ArrayAdding.java
Starting: runGood
```

```
Starting: runBad
$
```

As per the comment in the code, a system that fails once in a while will eventually kill someone. If you don't believe this, check out the Therac-25, in which a similar race condition killed a few people being treated for cancer.

There are examples of the `synchronized` keyword in several recipes, including Recipe 15.4.

11.5 Simplifying Synchronization with Locks

Problem

You want a more flexible means of synchronizing threads than the `synchronized` keyword.

Solution

Use the `Lock` mechanism in `java.util.concurrent.locks`.

Discussion

Use the java.util.concurrent.locks package; its major interface is `Lock`. This interface has several methods for locking and one for unlocking. Here is the general pattern for using `Lock`:

```
Lock thelock = ....
try  {
      lock.lock( );
      // do the work that is protected by the lock
} finally {
      lock.unlock( );
}
```

The point of putting the `unlock()` call in the `finally` block is, of course, to ensure that it is not bypassed if an exception occurs. (The code may also include one or more `catch` blocks, as required by the work being performed.)

The improvement here, compared with the traditional synchronized methods and blocks, is that using a `Lock` actually looks like a locking operation! And, as I mentioned, several means of locking are available, shown in Table 11-1. The `tryLock()` and `lockInterruptibly()` methods in the `Lock` interface solve another issue with synchronized blocks: that they block a thread in an uninterruptible state while it is waiting for a lock. There is no way to attempt to enter a synchronized block with a timeout governing how long it should wait. This makes deadlock significantly more likely.

Table 11-1. Locking methods of the `Lock` class

Return type	Method	Meaning
void	`lock( )`	Get the lock, even if you have to wait until another thread frees it first
boolean	`tryLock( )`	Get the lock only if it is free right now
boolean	`tryLock(long time, TimeUnit units) throws InterruptedException`	Try to get the lock, but only wait for the length of time indicated
void	`lockInterruptibly( ) throws InterruptedException`	Get the lock, waiting unless interrupted
void	`unlock( )`	Release the lock

The `TimeUnit` class lets you specify the units for the amount of time specified, including `TimeUnit.SECONDS`, `TimeUnit.MILLISECONDS`, `TimeUnit.MICROSECONDS`, and `TimeUnit.NANOSECONDS`.

In all cases, the lock must be released with `unlock()` before it can be locked again.

11.6 Locking with One Writer, Many Readers

Problem

You have many threads that need to read some data, while one thread occasionally needs to write or update the same date.

Solution

Use a readers-writer lock, the `ReentrantReadWriteLock`.

Discussion

The standard `Lock` is useful in many applications, but depending on the application's requirements, other types of locks may be more appropriate. Applications with asymmetric load patterns may benefit from a common pattern called the *reader-writer lock*. I call the lock in this recipe a readers-writer lock to emphasize that there can be many readers but only one writer. It's actually a pair of interconnected locks; any number of readers can hold the read lock and read the data, as long as it's not being written (shared read access). A thread trying to lock the write lock, however, waits until all the readers are finished and then locks them out until the writer is finished (exclusive write access). To support this pattern, both the `ReadWriteLock` interface and the implementing class `ReentrantReadWriteLock` are available. The interface has only two methods, `readLock()` and `writeLock()`, which provide a reference to the appropriate `Lock` implementation. *These methods do not, in themselves, lock or unlock the locks*; they only provide access to them, so it is common to see code like this:

```
rwlock.readLock( ).lock( );
...
rwlock.readLock( ).unlock( );
```

To demonstrate `ReadWriteLock` in action, I wrote the business logic portion of a web-based voting application. It could be used in voting for candidates or for the more common web poll. Presuming that you display the results on the home page and change the data only when somebody takes the time to click a response to vote, this application fits one of the intended criteria for `ReadWriteLock`—that is, that you have more readers than writers. The main class, `ReadersWritersDemo`, is shown in Example 11-8. The helper class `BallotBox` is online; it simply keeps track of the votes and returns a read-only `Iterator` upon request. Note that in the `run()` method of the reading threads, you could obtain the iterator while holding the lock but release the lock before printing it. This allows greater concurrency and better performance, but it could (depending on your application) require additional locking against concurrent update.

Example 11-8. main/src/main/java/threads/ReadersWriterDemo.java

```
public class ReadersWriterDemo {
 private static final int NUM_READER_THREADS = 3;

 public static void main(String[] args) throws Exception {
  new ReadersWriterDemo().demo();
 }
```

```
/** Set this to true to end the program */
private volatile boolean done = false;

/** The data being protected. */
private final BallotBox theData;

/** The read lock / write lock combination */
private final ReadWriteLock lock = new ReentrantReadWriteLock();

/**
 * Constructor: set up some quasi-random initial data
 */
public ReadersWriterDemo() throws Exception {
  List<String> choicesList = new ArrayList<>();
  choicesList.add("Agree");
  choicesList.add("Disagree");
  choicesList.add("No opinion");
  theData = new BallotBox(choicesList);
}

/**
 * Run a demo with more readers than writers
 */
private void demo() {

  // Start two reader threads
  for (int i = 0; i < NUM_READER_THREADS; i++) {
    Thread.startVirtualThread(() -> {
      while (!done) {
        lock.readLock().lock();
        try {
          theData.forEach(p ->
            System.out.printf("%s: votes %d%n",
              p.getName(),
              p.getVotes()));
        } finally {
          // Unlock in "finally" to be sure it gets done.
          lock.readLock().unlock();
        }

        try {
          Thread.sleep(((long)(Math.random() * 1000)));
        } catch (InterruptedException ex) {
          // nothing to do
        }
```

```
        }
      });
    }

    // Start one writer thread to simulate occasional voting
    Thread.startVirtualThread(() -> {
      while (!done) {
        lock.writeLock().lock();
        try {
          theData.voteFor(
            // Vote for random candidate :-)
            // Performance: should have one PRNG per thread.
            (((int)(Math.random() *
              theData.getCandidateCount())));
        } finally {
          lock.writeLock().unlock();
        }
        try {
          Thread.sleep(((long)(Math.random()*1000)));
        } catch (InterruptedException ex) {
          // nothing to do
        }
      }
    });

    // In the main thread, wait a while then terminate the run.
    try {
      Thread.sleep(10 * 1000);
    } catch (InterruptedException ex) {
      // nothing to do
    } finally {
      done = true;
    }
  }
}
```

Because this is a simulation and the voting is random, it does not always come out 50/50. In two consecutive runs, the following were the last line of each run:

```
Agree(6), Disagree(6)
Agree(9), Disagree(4)
```

See Also

The `Lock` interface also makes available `Condition` objects, which provide more flexibility. Consult the online documentation for more information.

It's a bit unfortunate that these locks are not `AutoCloseable`. One might wrap them in a subclass or delegation with `close()` delegating to `unlock()`. See this StackOverflow post for more information.

11.7 Sharing Data Among Threads—ThreadLocal and ScopedValue: Structuring Concurrency

Problem

You need to share data among multiple threads.

Solution

Use a `ThreadLocal`, or, in Java 21+ preview, use a `ScopedValue`.

Discussion

In Java, all *nonlocal* data is shared between threads. The `ThreadLocal` class, in Java since 1.3, provides a way to hide data such that there is an instance for each thread, by effectively providing a `Map` from the current thread to an arbitary object, giving the illusion of each thread having its own copy of a given variable. Suppose we have multiple threads, each reading records sequentially from a shared data store. We could write this two ways, using either aggregation or inheritance. The example in Example 11-9 uses inheritance, subclassing `ThreadLocal` to provide an `initialValue()` method; this style is preferred.

Example 11-9. main/src/main/java/threads/ThreadLocalDemo.java

```
/** This ThreadLocal holds the Client reference for each Thread.
 * Make ThreadLocal instance static, to show that it is not an instance variable
 * but is derived from Thread.currentThead() regardless of static/instance access.
 */
private static ThreadLocal<Client> myClient = new ThreadLocal<Client>() {
 // initialValue() is called magically when you first call get().
```

```
    @Override
    protected synchronized Client initialValue() {
      return new Client(++clientNum);
    }
  };
  // IRL this method would do something useful.
  @Override
  public void run() {
    System.out.println("Thread " + Thread.currentThread().getName() +
      " sees client " + myClient.get());
  }

  public static void main(String[] args) {
    new ThreadLocalDemo("demo 1").start();
    new ThreadLocalDemo("demo 2").start();
    Thread.yield();
    System.out.println("Main program sees client " + myClient.get());
  }

  /** A serial number for clients */
  private static int clientNum = 0;

  /** Simple data class, in real life clients would have more fields! */
  private static record Client(int clientNum) { }
```

Because threading is necessarily indeterminate about the order in which code will run, this produces one of several different outputs when run:

```
$ java ThreadLocalDemo.java
Main program sees client Client[clientNum=2]
Thread demo 2 sees client Client[clientNum=3]
Thread demo 1 sees client Client[clientNum=1]
$ r
java ThreadLocalDemo.java
Thread demo 1 sees client Client[clientNum=1]
Main program sees client Client[clientNum=2]
Thread demo 2 sees client Client[clientNum=3]
$
```

If you change the code to use an `AtomicInteger` and remove the `protected` attribute from `initialValue()`, the code becomes fast enough that it appears deterministic—`main()` always wins. This is done in *ThreadLocalDemo2.java* in the same directory.

ThreadLocalDemo3.java demonstrates using explicit creation of the object via aggregation rather than subclassing `ThreadLocal`.

21P Java 21 introduces a preview class called `ScopedValue`. This class provides access to a value in the thread it's associated with, and, importantly, in any method called by that thread. In effect, this allows for the passing of data to a method multiple levels down in the call stack without adding it as parameter(s) to every method along the way.

As a simple example:

```
static final ScopedValue<Client> CLIENT = ScopedValue.newInstance();
private final Client client = new Client().
ScopedValue.runWhere(CLIENT, client, () -> doSomething());
```

Assuming `doSomething()` calls `doSomethingElse()`, which in turn calls `doYetMore()`, the latter can find the client object by calling:

```
Client client = client.GET();
```

A runnable version of this code is in Example 11-10.

Example 11-10. main/src/main/java/threads/ScopedValueDemo1.java

```
static final ScopedValue<Client> CLIENT = ScopedValue.newInstance();

private final Client currentClient = new Client(1234);

void main() {
 ScopedValue.runWhere(CLIENT, currentClient, () -> doSomething());
}

void doSomething() {
 doSomethingElse();
}
void doSomethingElse() {
 doYetMore();
}
void doYetMore() {
 Client client = CLIENT.get();
 System.out.println("In doYetMore, client = " + client);
}
```

```
record Client(int id) { }
```

The `ScopedValue` object is bound when the `runWhere` method begins, and unbound when it exits.

Note that a `ScopedValue` is unsurprisingly tied to a given thread. To pass a value to multiple threads you need a `StructuredTaskScope` to bind the value in multiple threads. Such objects must be immutable to prevent race conditions, or an appropriate synchronization means must be used. Suppose we wanted to run three methods each in a separate thread. We simply wrap all the methods in the `StructuredTaskScope`, as shown in Example 11-11.

Example 11-11. main/src/main/java/threads/StructuredTaskScopedemo.java

```
private static final ScopedValue<Client> CLIENT = ScopedValue.newInstance();

private final Client currentClient = new Client(1234);

void main() {

  ScopedValue.runWhere(CLIENT, currentClient, () -> {
    try (var scope = new StructuredTaskScope<Object>()) {
      scope.fork(doSomething);
      scope.fork(doSomethingElse);
      scope.fork(doYetMore);
      scope.join();
    } catch (InterruptedException iex) {
      System.out.println("Uncool! You interrupted me.");
    }
  });
}

Callable<String> doSomething = () -> {
  System.out.println("In doSomething, client " + CLIENT.get());
  return "One";
};
Callable<String> doSomethingElse = () -> {
  System.out.println("In doSomethingElse, not much here.");
  return "Two";
};
Callable<Integer> doYetMore = () -> {
  System.out.println("In doYetMore, using client " + CLIENT.get());
  return 42;
```

```
};

record Client(int id) { }
```

The `join()` method waits for completion of all tasks, and the `close()` method abandons the task scope. It's a mistake to call the former without the latter. The scope also has a `shutdown()` method to shut down an entire task scope without closing it; this method *cancels all unfinished subtasks* by interrupting the threads, and will not start any new threads. If the scope's owner is waiting in the `join()` method then it will wake up.

As for when to use `ThreadLocal` versus `ScopedValue`, the documentation states:

> *"A `ScopedValue` should be preferred over a `ThreadLocal` for cases where the goal is "one-way transmission" of data without using method parameters. While a `ThreadLocal` can be used to pass data to a method without using method parameters, it does suffer from a number of issues:*
>
> - *`ThreadLocal` does not prevent code in a faraway callee from setting a new value.*
> - *A `ThreadLocal` has an unbounded lifetime and thus continues to have a value after a method completes, unless explicitly removed.*
> - *Inheritance is expensive—the map of thread-locals to values must be copied when creating each child thread."*

See Also

The ideas behind structured concurrency are detailed in JEP 428.

11.8 Simplifying Producer/Consumer with the Queue Interface

Problem

You need to control producer/consumer implementations involving multiple threads.

Solution

Use the `Queue` interface or the `BlockingQueue` subinterface.

Discussion

As an example of the simplifications possible with the `java.util.Concurrent` package, consider the standard producer/consumer program. An implementation synchronized using traditional `Thread` code (`wait()` and `notifyAll()`) is in the online source as `ProdConsThreadAPI` (its code is not shown here). Example 11-12 uses the `java.util.BlockingQueue` (a subinterface of `java.util.Queue`) to reimplement `ProdConsThreadAPI` in about two-thirds the number of lines of code, and it's simpler. The application simply puts items into a queue and takes them from it. In the example, I have four producers and only three consumers, so the producers eventually wait. Running the application on one of my older notebooks, the producers' lead over the consumers increases to about 350 over the 10 seconds or so of running it.

Another way of handling producer/consumer is with the `Semaphore` class in `java.util.concurrent` . Semaphores were designed by famous computer scientist Edsger Dijkstra to provide a thread-safe way of maintaining a count of available resources. This is not expanded on in the book, but there is an implementation online in *main/src/main/java/threads/ProdConsSemaphore.java*.

Example 11-12. main/src/main/java/threads/ProdConsQueues.java

```
public class ProdConsQueues {

  protected volatile boolean done = false;
  protected ExecutorService threadPool =
    Executors.newCachedThreadPool();

  /** Inner class representing the Producer side */
  class Producer implements Runnable {

    protected BlockingQueue<Object> queue;

    Producer(BlockingQueue<Object> theQueue) { this.queue = theQueue; }

    public void run() {
      try {
        while (!done) {
          Object justProduced = getRequestFromNetwork();
          queue.put(justProduced);
          System.out.println(
            "Produced 1 object; List size now " + queue.size());
        }
      } catch (InterruptedException ex) {
```

```
      System.out.println("Producer INTERRUPTED");
    }
  }

  Object getRequestFromNetwork() {  // Simulation of reading from client
    try {
       Thread.sleep(10); // simulate time passing during read
    } catch (InterruptedException ex) {
      System.out.println("Producer Read INTERRUPTED");
    }
    return new Object();
  }
}

/** Inner class representing the Consumer side */
class Consumer implements Runnable {
  protected BlockingQueue<Object> queue;

  Consumer(BlockingQueue<Object> theQueue) { this.queue = theQueue; }

  public void run() {
    try {
      while (true) {
        Object obj = queue.take();
        int len = queue.size();
        System.out.println("List size now " + len);
        process(obj);
        if (done) {
          return;
        }
      }
    } catch (InterruptedException ex) {
       System.out.println("CONSUMER INTERRUPTED");
    }
  }

  void process(Object obj) {
    // Thread.sleep(123) // Simulate time passing
    System.out.println("Consuming object " + obj);
  }
}

ProdConsQueues(int nP, int nC) {
  BlockingQueue<Object> myQueue = new LinkedBlockingQueue<>();
  for (int i=0; i<nP; i++)
    threadPool.submit(new Producer(myQueue));
```

```
    for (int i=0; i<nC; i++)
      threadPool.submit(new Consumer(myQueue));
  }

  public static void main(String[] args)
    throws IOException, InterruptedException {

    // Start producers and consumers
    int numProducers = 4;
    int numConsumers = 3;
    ProdConsQueues pc = new ProdConsQueues(numProducers, numConsumers);

    // Let the simulation run for, say, 10 seconds
    Thread.sleep(10*1000);

    // End of simulation - shut down gracefully
    pc.done = true;
    pc.threadPool.shutdown();
  }
}
```

`ProdConsQueues` is superior to `ProdConsThreadAPI` in almost all aspects. However, the queue sizes that are output no longer necessarily reflect the size of the queue after the object is inserted or removed. Because there's no longer any locking ensuring atomicity here, any number of queue operations could occur on other threads between the `Producer` thread's `queue.put()` and the `Consumer` thread's queue size query.

11.9 Optimizing Parallel Processing with Fork/Join

Problem

You want to optimize use of multiple processors and/or large problem spaces.

Solution

Use the Fork/Join framework.

Discussion

Fork/Join is an `ExecutorService` intended mainly for reasonably large tasks that can naturally be divided recursively, where you don't have to ensure equal timing for each division. It uses work-stealing to keep threads busy.

The basic means of using Fork/Join is to extend `RecursiveTask` or `RecursiveAction` and override its `compute()` method along these lines:

```
if (assigned portion of work is "small enough") {
      perform the work myself
} else {
      split my work into two pieces
      invoke the two pieces and await the results
}
```

There are two classes: `RecursiveTask` and `RecursiveAction`. The main difference is that `RecursiveTask` has each step of the work returning a value, whereas `RecursiveAction` does not. In other words, the `RecursiveAction` method `compute()` has a return type of `void`, whereas the `RecursiveTask` method of the same name has a return type of `T`, some type parameter. You might use `RecursiveTask` when each call returns a value that represents the computation for its subset of the overall task, in other words, to divide a problem like summarizing data—each task would summarize one part and return that. You might use `RecursiveAction` to operate over a large data structure performing some transform of the data in place.

There are two demos of the Fork/Join framework here, named after the `ForkJoinTask` that each subclasses:

- `RecursiveTaskDemo` uses `fork()` and `join()` directly.
- `RecursiveActionDemo` uses `invokeAll()` to invoke the two subtasks. `invoke()` is just a `fork()` and a `join()`; and `invokeAll()` just does this repeatedly until done. Compare the versions of `compute()` in Examples 11-13 and 11-14 and this will make sense.

Example 11-13. main/src/main/java/threads/RecursiveActionDemo.java

```
/** A trivial demonstration of the "Fork/Join" framework:
 * square a bunch of numbers using RecursiveAction.
 * We use RecursiveAction here b/c we don't need each
 * compute() call to return its result; the work is
 * accumulated in the "dest" array.
```

```
 * @see RecursiveTaskDemo when each computation has to return a value.
 */
public class RecursiveActionDemo extends RecursiveAction {

 @Serial
 private static final long serialVersionUID = 3742774374013520116L;

 static int[] raw = {
  19, 3, 0, -1, 57, 24, 65, Integer.MAX_VALUE, 42, 0, 3, 5
 };
 static int[] sorted = null;

 int[] source;
 int[] dest;
 int length;
 int start;
 final static int THRESHOLD = 4;

 public static void main(String[] args) {
  sorted = new int[raw.length];
  RecursiveActionDemo fb =
   new RecursiveActionDemo(raw, 0, raw.length, sorted);
  ForkJoinPool pool = new ForkJoinPool();
  pool.invoke(fb);
  System.out.print('[');
  for (int i : sorted) {
   System.out.print(i + ",");
  }
  System.out.println(']');
  pool.close();
 }

 public RecursiveActionDemo(int[] src, int start, int length, int[] dest) {
    this.source = src;
    this.start = start;
    this.length = length;
    this.dest = dest;
  }

 @Override
 protected void compute() {
  System.out.println("RecursiveActionDemo.compute()");
  if (length <= THRESHOLD) { // Compute Directly
   for (int i = start; i < start + length; i++) {
    dest[i] = source[i] * source[i];
   }
```

```
    } else {            // Divide and Conquer
      int split = length / 2;
      invokeAll(
        new RecursiveActionDemo(source, start,       split,        dest),
        new RecursiveActionDemo(source, start + split, length - split, dest));
    }
  }
}
```

Example 11-14. main/src/main/java/threads/RecursiveTaskDemo.java

```
/**
 * Demonstrate the Fork/Join Framework to average a large array.
 * Running this on a multi-core machine as e.g.,
 * $ time java threads.RecursiveTaskDemo
 * shows that the CPU time is always greater than the elapsed time,
 * indicating that we are making use of multiple cores.
 * That said, it is a somewhat contrived demo.
 *
 * Use RecursiveTask<T> where, as in this example, each call returns
 * a value that represents the computation for its subset of the overall task.
 * @see RecursiveActionDemo when each computation does not return a value,
 * e.g., when each is just working on some section of a large array.
 */
public class RecursiveTaskDemo extends RecursiveTask<Long> {

  @Serial
  private static final long serialVersionUID = 74277437329822011L;

  static final int N = 10000000;
  final static int THRESHOLD = 500;

  int[] data;
  int start, length;

  public static void main(String[] args) {
    int[] source = new int[N];
    loadData(source);
    RecursiveTaskDemo fb = new RecursiveTaskDemo(source, 0, source.length);
    ForkJoinPool pool = new ForkJoinPool();
    long before = System.currentTimeMillis();
    pool.invoke(fb);
    long after = System.currentTimeMillis();
    long total = fb.getRawResult();
    long avg = total / N;
```

```
    System.out.println("Average: " + avg);
    System.out.println("Time :" + (after - before) + " mSec");
    pool.close();
  }

  static void loadData(int[] data) {
    Random r = new Random();
    for (int i = 0; i < data.length; i++) {
      data[i] = r.nextInt();
    }
  }

  public RecursiveTaskDemo(int[] data, int start, int length) {
     this.data = data;
     this.start = start;
     this.length = length;
  }

  @Override
  protected Long compute() {
    if (length <= THRESHOLD) { // Compute directly
      long total = 0;
      for (int i = start; i < start + length; i++) {
        total += data[i];
      }
      return total;
    } else {            // Divide and conquer
      int split = length / 2;
      RecursiveTaskDemo t1 =
        new RecursiveTaskDemo(data, start,       split);
      t1.fork();
      RecursiveTaskDemo t2 =
        new RecursiveTaskDemo(data, start + split, length - split);
      return t2.compute() + t1.join();
    }
  }
}
```

The biggest undefined part is "small enough"; you may have to do some experimentation to see what works well as a chunk size. Or, better yet, write more code using a feedback control system, measuring the system throughput as the parameter is dynamically tweaked up and down, and have the system automatically arrive at the optimal value for that particular computer system and runtime. This is left as an extended exercise for the reader.

The Fork/Join pool is also used in the implementation of alternative concurrency models such as the actor model (as implemented in Akka). It is more general than the recursive task decomposition class of problems. One of its most attractive features is work-stealing, where a task queued for execution on one thread can be taken by another if it is felt that this would result in it being completed sooner.

11.10 Scheduling Tasks: Future Times, Background Saving in an Editor

Problem

You need to schedule something for a fixed time in the future. You need to save the user's work periodically in an interactive program.

Solution

For one-shot future tasks, use the `Timer` service with a `TimerTask` object. For recurring tasks, either use a background thread, or use the `Timer` service and recompute the next time. For more complex tasks, such as running something at high noon every second Thursday, consider using a third-party scheduling library such as Quartz or, in JavaEE/Jakarta, the EJB Timer Service.

Discussion

There are several ways of scheduling things in the future. For one-shot scheduling, you can use the `Timer` service from `java.util`. For recurring tasks, you can use a `Runnable`, which sleeps in a loop.

Here is an example of the `Timer` service in `java.util`. These are the basics of using this API:

1. Create a `Timer` service object.
2. Use it to schedule instances of `TimerTask` with a legacy `Date` object[1] indicating the date and time when the task should be invoked.

The code in Example 11-15 subclasses `TimerTask` as `Item` to perform a simple notification action in the future, based on reading lines with `year-month-day-hour-minute Task`, such as the following:

```
2025 12 25 10 30 Get some sleep.
2025 12 26 01 27 Finish this program
2025 12 25 01 29 Document this program
```

Example 11-15. main/src/main/java/threads/ReminderService.java

```
public class ReminderService {

 /** The Timer object */
 Timer timer = new Timer();

 class Item extends TimerTask {
  String message;
  Item(String m) {
   message = m;
  }
  public void run() {
   message(message);
  }
 }

 public static void main(String[] argv) throws Exception {
  new ReminderService().loadReminders();
 }

 private String dfPattern = "yyyy MM dd hh mm ss";
 private SimpleDateFormat formatter = new SimpleDateFormat(dfPattern);

 protected void loadReminders() throws Exception {

  Files.lines(Path.of("ReminderService.txt")).forEach(aLine -> {

   ParsePosition pp = new ParsePosition(0);
   Date date = formatter.parse(aLine, pp);
   String task = aLine.substring(pp.getIndex());
   if (date == null) {
    System.out.println("Invalid date in " + aLine);
    return;
   }
   System.out.println("Date = " + date + "; task = " + task);
   timer.schedule(new Item(task), date);
  });
 }
```

In real life the program would need to run for long periods of time and use a more sophisticated messaging pattern; here we only show the timing scheduling portion.

The code fragment in Example 11-16 creates a background thread to handle background saves, as in most word processors.

Example 11-16. main/src/main/java/threads/AutoSave.java

```
public class AutoSave implements Runnable {
  /** The FileSave interface is implemented by the main class. */
  protected FileSaver model;
  /** How long to sleep between tries */
  public static final int MINUTES = 5;
  private static final int SECONDS = MINUTES * 60;

  public AutoSave(FileSaver m) {
    Thread.currentThread().setName("AutoSave Thread");
    Thread.currentThread().setDaemon(true);    // so we don't keep the main app alive
    model = m;
  }

  public void run() {
    while (true) {    // entire run method runs forever.
      try {
        Thread.sleep(SECONDS*1000);
      } catch (InterruptedException e) {
        // do nothing with it
      }
      if (model.wantAutoSave() && model.hasUnsavedChanges())
        model.saveFile(null);
    }
  }

  // Not shown:
  // 1) saveFile() must now be synchronized.
  // 2) method that shuts down main program must be synchronized on *SAME* object
}

/** Local copy of FileSaver interface, for compiling AutoSave demo. */
interface FileSaver {
  /** Load new model from fn; if null, prompt for new fname */
  public void loadFile(String fn);

  /** Ask the model if it wants AutoSave done for it */
  public boolean wantAutoSave();
```

```
    /** Ask the model if it has any unsaved changes, don't save otherwise */
    public boolean hasUnsavedChanges();

    /** Save the current model's data in fn.
     * If fn == null, use current fname or prompt for a filename if null.
     */
    public void saveFile(String fn);
}
```

As you can see in the `run()` method, this code sleeps for five minutes (300 seconds), then checks whether it should do anything. If the user has turned autosave off, or hasn't made any changes since the last save, nothing needs to be done. Otherwise, we call the `saveFile()` method in the main program; this saves the data to the current file. It would be smarter to save it to a recovery file of some name, as the better word processors do.

What's not shown is that now all the methods must be synchronized. It's easy to see why if you think about how the save method would work if the user clicked the Save button at the same time that the autosave method called it, or if the user clicked Exit while the file save method had just opened the file for writing. The strategy of saving to a recovery file gets around some of this, but it still needs a great deal of care.

See Also

For details on `java.util.concurrent`, see the documentation accompanying the JDK. For background on JSR-166, see Doug Lea's home page and his JSR-166 page.

A great reference on Java threading is *Java Concurrency in Practice* by Brian Goetz et al. (Addison-Wesley).

Project Loom: Fibers and Continuations produced the virtual threads implementation and, further, aims to promote easier-to-use, lighter-weight concurrency mechanisms.

1 It's unfortunate that this API has not been updated to the "new" (in Java 8) date/time API.

‹ 10. Input and Output: Reading, Writing, and Directory Tricks

12. Data Science and R ›